4 Generalised Linear Models

4.1 Introductory remarks

- Let $f(\boldsymbol{x})$ be a non-linear feature mapping
- Aim: Models with nonlinear mapping of feature vectors into a transformation space
- Kernel functions [Aizerman et al., 1964] [Boser et al., 1992] $k: \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$
  - Symmetric functions: $k(\boldsymbol{x}, \boldsymbol{x}') = k(\boldsymbol{x}', \boldsymbol{x})$
  - Important: due to the Mercer theorem, kernel functions can be represented by an inner product (1)
  - The so-called **kernel trick** means the following substitution: if an algorithm applies inner products to mappings of feature vectors $\boldsymbol{x}$, then we can replace the inner product by any choice of a kernel function.
  - This way, we have an implicit nonlinear mapping without determining the mapping explicitly.
- Methods, where the kernel trick has been applied, are:
  - Perceptron networks [Aizerman et al., 1964; Friend & Schapire, 1999]
  - Support vector machines (SVM) [Boser et al., 1992],
  - Principal component analysis (PCA) [Schölkopf et al., 1998],
  - Fisher LDA [Mika et al., 1999; Roth & Steinhage, 2000; Baudat & Anouar, 2000],
  - Nearest neighbour classifiers [Baudat & Anouar, 2003],
  - Several methods of regression and of cluster analysis.

4.2 Dual Representations

- Many **linear models of regression and classification** can be expressed by dual representation using **kernel functions**
- Given: generalised linear regression model

(4.1)

- The solution $\boldsymbol{w}^*$ is searched by minimising the regularised sum of squared errors as objective function:

(4.2)

- **Solution**:

(4.3)

  - Linear combination of the vectors $\boldsymbol{f}(\boldsymbol{x})$
  - Represented by the so-called **design matrix** $\boldsymbol{F}$, which contains $\boldsymbol{f}(\boldsymbol{x})$ as row vectors
  - Linear factors are by vector $\boldsymbol{a} \in \mathbb{R}^N$ having the following components:

(4.4)

  - **Dual representation**: reformulation of eq. (4.2), in which $\boldsymbol{w} = \boldsymbol{F}^{\mathrm{T}}\boldsymbol{a}$ and $\boldsymbol{t} \in \mathbb{R}^N$ are used (see exercise)

$$J(\boldsymbol{a}) = \frac{1}{2}\,\boldsymbol{a}^{\mathrm{T}}\,\boldsymbol{F}\,\boldsymbol{F}^{\mathrm{T}}\,\boldsymbol{F}\,\boldsymbol{F}^{\mathrm{T}}\,\boldsymbol{a} - \boldsymbol{a}^{\mathrm{T}}\,\boldsymbol{F}\,\boldsymbol{F}^{\mathrm{T}}\,\boldsymbol{t} + \frac{1}{2}\,\boldsymbol{t}^{\mathrm{T}}\,\boldsymbol{t} + \frac{\lambda}{2}\,\boldsymbol{a}^{\mathrm{T}}\,\boldsymbol{F}\,\boldsymbol{F}^{\mathrm{T}}\,\boldsymbol{a} \tag{4.5}$$

  - Introduction of the **Gram matrix**: (2)
    which is a $N \times N$ symmetric matrix
    having the following elements: (3)
  - (4.5) can be reformulated (the way from (4.1) to (4.6) will be demonstrated during the exercise)

$$J(\boldsymbol{a}) = \frac{1}{2}\,\boldsymbol{a}^{\mathrm{T}}\,\boldsymbol{K}\,\boldsymbol{K}\,\boldsymbol{a} - \boldsymbol{a}^{\mathrm{T}}\,\boldsymbol{K}\,\boldsymbol{t} + \frac{1}{2}\,\boldsymbol{t}^{\mathrm{T}}\,\boldsymbol{t} + \frac{\lambda}{2}\,\boldsymbol{a}^{\mathrm{T}}\,\boldsymbol{K}\,\boldsymbol{a} \tag{4.6}$$

- The solution is found by minimising (4.6) (to find (4.7) will be demonstrated during the exercise)
  - First derivative with respect to $\boldsymbol{a}$
  - Setting the derivative equal to zero
  - Dissolve to $\boldsymbol{a}$
- **Solution**:

(4.7)

- This gives the following expression of the output variable

(4.8)

  - Where $\boldsymbol{k}(\boldsymbol{x}) = \boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{f}(\boldsymbol{x}_n)$ is a vector of kernel functions having the following components: $k(\boldsymbol{x}, \boldsymbol{x}_n)$
- This way, we have shown that
  we can express the least squares (LS) regression completely with kernel functions
- Later: this way, we will utilise kernels for generating high-dimensional transform spaces

4.3 Generation of Kernel Functions

- **Simple approach**: choose a set of nonlinear functions $f_i(\boldsymbol{x})$ and construct the kernel function with it

(4.9)

  - **Alternative approach**: direct generation, but a check for validity is required

  - Example in $\mathbb{R}^2$:
    $\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$
    $\boldsymbol{x}' = \begin{pmatrix} x_1' \\ x_2' \end{pmatrix}$

    $\rightarrow$ The mapping contains all 2nd order monomials with specific weights
  - General and simple test [Shawe-Taylor & Cristianini, 2004]
    - Gram matrix $\boldsymbol{K}$ must be positive semidefinite: $\boldsymbol{x}^{\mathrm{T}} \boldsymbol{K} \boldsymbol{x} \geq 0 \;\; \forall \, \boldsymbol{x} \in \mathbb{R}^d$
- **Further approach**: use already known kernel functions as building blocks
  - Given are two admissible kernel functions $k_1(\boldsymbol{x}, \boldsymbol{x}'), k_2(\boldsymbol{x}, \boldsymbol{x}')$
  - Let $f(\boldsymbol{x})$: $\mathbb{R}^d \mapsto \mathbb{R}$ be a continuous function, let $c \in \mathbb{R}$ be a constant value
  - Let $\boldsymbol{f}(\boldsymbol{x})$: $\mathbb{R}^d \mapsto \mathbb{R}^D$ be a continuous f., let $p(y)$: $\mathbb{R} \mapsto \mathbb{R}$ by a polynomial with non-negative coefficients
  - From them, further valid kernel functions can be constructed:

$$
\begin{aligned}
&(1)\quad k(\boldsymbol{x}, \boldsymbol{x}') = c\, k_1(\boldsymbol{x}, \boldsymbol{x}') && (2)\quad k(\boldsymbol{x}, \boldsymbol{x}') = f(\boldsymbol{x})\, k_1(\boldsymbol{x}, \boldsymbol{x}')\, f(\boldsymbol{x}') \\
&(3)\quad k(\boldsymbol{x}, \boldsymbol{x}') = p\big(k_1(\boldsymbol{x}, \boldsymbol{x}')\big) && (4)\quad k(\boldsymbol{x}, \boldsymbol{x}') = \exp\big(k_1(\boldsymbol{x}, \boldsymbol{x}')\big) \\
&(5)\quad k(\boldsymbol{x}, \boldsymbol{x}') = k_1(\boldsymbol{x}, \boldsymbol{x}') + k_2(\boldsymbol{x}, \boldsymbol{x}') && (6)\quad k(\boldsymbol{x}, \boldsymbol{x}') = k_1(\boldsymbol{x}, \boldsymbol{x}')\, k_2(\boldsymbol{x}, \boldsymbol{x}') \\
&(7)\quad k(\boldsymbol{x}, \boldsymbol{x}') = k_1\big(\boldsymbol{f}(\boldsymbol{x}), \boldsymbol{f}(\boldsymbol{x}')\big) && (8)\quad k(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{x}^{\mathrm{T}} \boldsymbol{A}\, \boldsymbol{x}'
\end{aligned}
\tag{4.10}
$$

  - (3), (8): Polynomial kernel functions $k(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x} \cdot \boldsymbol{x}')^2$ containing only 2nd order monomials
  - (3), (8): Generalized form $k(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x} \cdot \boldsymbol{x}' + c)^2$ containing monomials of 0th, 1st, and 2nd order
  - (3), (8): Polynomial kernel functions $k(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x} \cdot \boldsymbol{x}')^d$ containing all monomials of the order d
  - (3), (8): Generalized form $k(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x} \cdot \boldsymbol{x}' + c)^d$ containing all monomials of order 0, 1, up to d
  - Gaussian kernel function (which has no pre-factor because it must not be a pdf)

(4.11)

- Derivation of the squared function: due to (2) and (4) in (4.10) and due to the validity of linear kernel functions $\kappa(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{x} \cdot \boldsymbol{x}'$ the following results:

$$\|\boldsymbol{x} - \boldsymbol{x}'\|^2 = (\boldsymbol{x} - \boldsymbol{x}') \cdot (\boldsymbol{x} - \boldsymbol{x}') = \boldsymbol{x} \cdot \boldsymbol{x} - 2\, \boldsymbol{x} \cdot \boldsymbol{x}' + \boldsymbol{x}' \cdot \boldsymbol{x}'$$

$$(4.11) \quad \rightarrow \quad k(\boldsymbol{x}, \boldsymbol{x}') = \exp\left(-\frac{\boldsymbol{x} \cdot \boldsymbol{x}}{2\,\sigma^2}\right) \exp\left(-\frac{\boldsymbol{x} \cdot \boldsymbol{x}'}{\sigma^2}\right) \exp\left(-\frac{\boldsymbol{x}' \cdot \boldsymbol{x}'}{2\,\sigma^2}\right) = f(\boldsymbol{x}) \exp\big(c\; k_1(\boldsymbol{x}, \boldsymbol{x}')\big)\; f(\boldsymbol{x}')$$

- **Important**: the function vector in (4.9) which corresponds to the Gaussian kernel function is of infinite dimensionality ($D \rightarrow \infty$); there are infinitely man eigenfunctions f_i and eigenvalues λ_i
- **Theorem of Mercer**: $k(\boldsymbol{x}, \boldsymbol{x}') = \sum_{i=1}^{D} \lambda_i\, f_i(\boldsymbol{x})\, f_i(\boldsymbol{x}') = \boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{f}(\boldsymbol{x}')$
- Squared Euclidean distance between $\boldsymbol{x}, \boldsymbol{x}'$ can be replaced by the nonlinear kernel $k(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{x} \cdot \boldsymbol{x}'$:

$$\|\boldsymbol{x} - \boldsymbol{x}'\|^2 = k(\boldsymbol{x}, \boldsymbol{x}) + k(\boldsymbol{x}', \boldsymbol{x}') - 2\, k(\boldsymbol{x}, \boldsymbol{x}')$$

- Likewise, kernel function approaches can also be formed for symbolic features and for objects such as graphs, quantities, strings, text documents, probability density functions.
- Sigmoidal kernel function [Vapnik, 1995] with free parameters $a, b \in \mathbb{R}$

(4.12)

- Gram-Matrix of sigmoidal kernels is however not positive semidefinite

4.4 Theorem of Cover

- Separability of feature vectors [Cover, 1965]
  A nonlinear separable classification problem is linear separable with higher probability in a higher than in a lower dimensional transform space, if the space is not densely occupied.
- The last part of Cover's theorem is not of high importance
  - Densely occupied spaces are not important in practise, as the sizes of the data sets are always too small
  - Densely occupied spaces are currently not important in theory, since we are able to construct infinitely dimensional transform spaces
- Given
  - Training set: (1)
  - Two class regions (two domains): (2)
  - Transform vector $\boldsymbol{f}(\boldsymbol{x})$ containing D different nonlinear functions: (3)
- **Dichotomy**: Every separation function of a sample $\mathcal{S}$ which generates two non-empty subsets of $\mathcal{S}$
- Any dichotomy $\{G_1, G_2\}$ is *f*-separable if a vector $\boldsymbol{w} \in \mathbb{R}^D$ exists, such that the following holds:

(4.13)

- I.e. the regions are linearly separable in the *f*-space with **separation function**: $\boldsymbol{w} \cdot \boldsymbol{f}(\boldsymbol{x}) = 0$
- A corresponding class of relatively simple mappings is obtained by linear combinations of monomials (product terms in the coordinates) of order *r*
- That is, the separation function is a linear combination of monomials:

(4.14)

- Within the *d*-dimensional space there exist *q* different monomials:

(4.15)

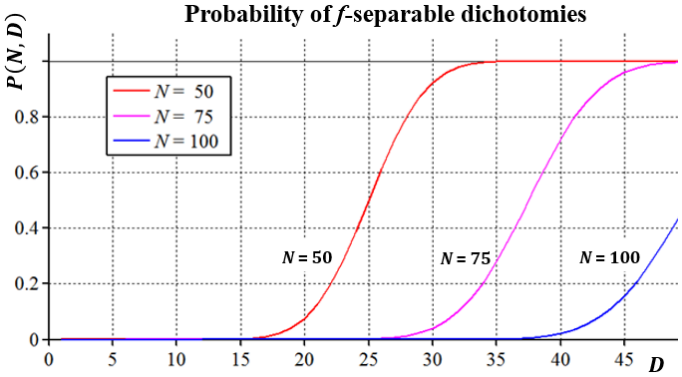
- Separation functions (4.14) are polynomial functions: hyperplanes, hyperellipsoids, hyperparaboloids, hyperhyperboloid, cubic hyperplanes, …

- Examples in $\mathbb{R}^2$: *f*-separable dichotomies
  a) Linear (1) b) quadratic (2)

- Random experiment: let $(\boldsymbol{x}_n, t_n)$ be randomly distributed and i.i.d.
  - Let the regions G_1, G_2 be equally probable:
    a priori probabilities are at 50 %
  - Then, the number of ***f*-separable dichotomies** is: (3)
  - And the number of **all dichotomies** is: (4)
  - Their ratio is equal to the **probability** $P(N, D)$ that a dichotomy is *f*-separable:

(4.16)

  - For $D = \frac{N}{2}$ we always have: $P(N, D) = \frac{1}{2}$



Probability $P(N, D)$ of *f*-separable dichotomies versus dimensionality D of the transform space for three different sample sizes N.

- Blessing of high dimensionality
  The higher the dimensionality D of the transform space, the higher is $P(N, D)$
- In infinite dimensional spaces, each dichotomy is linearly separable: $D \to \infty \quad \Rightarrow \quad P(N, D) = 1$
- **Separation capacity** of a separation function
  - If K is the maximum number of feature vectors that are *f*-separable,
    and K is considered a random variable, then if follows for the probability that $K = N$ holds:

(4.17)

  - Because f has D degrees of freedom in the D-dimensional space
  - It can be shown [Haykin, 2009] that (4.17) corresponds to a negative binomial distribution for binary random experiments with equal probability for both events, such that:
    The expected maximum number of *f*-separable vectors in a *D*-dimensional space is equal to $2D$

4.5 Radial Basis Function Networks (RBF nets)

Regression with linear combination of fixed RBF
- **RBF** only depend on the radial distance between $\boldsymbol{x}$ and a centroid vector $\boldsymbol{\mu}_n$ such that:

(4.18)

- Given: sample (1)

- Sought: continuous function $h(\boldsymbol{x})$ approximating the given target values t_n for all given feature vectors $\boldsymbol{x}_n$: (2)

- Approach: linear combination of RBF [Powell, 1987]:

(4.19)

- The coefficients w_n are found with least squares minimization

Interpolation: Because the number of coefficients is the same as the number of constraints, a function must be found that runs exactly through each target value: (3)

- In practice, the target values and the feature vectors have random noise, so that an interpolation solution is undesirable (overfitting); i.e. we have mostly regression tasks

Structural Risk Minimisation (SRM): Regularised approach [Poggio & Girosi, 1990]
- By regularisation: no interpolation $h(\boldsymbol{x}_n) = t_n$, but regression $h(\boldsymbol{x}_n) \approx t_n$
- Optimal solution: approximation with eigenfunctions of a differential operator (Greens functions)
- **Nadaraya-Watson model**: at each feature vector $\boldsymbol{x}_n$ an RBF is centred
- However, this is computationally time-consuming and ineffective for large amounts of data
- It is more effective to find $K < N$ representative feature vectors (support vectors)
  → sparse kernel methods: support vector regression (later more)

Kernel Regression (Nadaraya-Watson Model) [Nadaraya 1964], [Watson, 1964]
- Model of the joint pdf $p(\boldsymbol{x}, t)$ with N density functions $f(\boldsymbol{x}, t)$ centered at each feature vector $\boldsymbol{x}_n \in S$

$$p(\boldsymbol{x}, t) = \frac{1}{N}\sum_{n=1}^{N} f(\boldsymbol{x} - \boldsymbol{x}_n, t - t_n) \tag{4.20}$$

- Regression function $h(\boldsymbol{x})$ represented by mean values of the target variable t with weighting by the pdf of t given $\boldsymbol{x}$:

$$h(\boldsymbol{x}) = \mathbb{E}(t|\boldsymbol{x}) = \int_{-\infty}^{\infty} t\, p(\boldsymbol{x}, t)\, dt = \frac{\int t\, p(\boldsymbol{x}, t)\, dt}{\int p(\boldsymbol{x}, t)\, dt} = \frac{\sum_n \int t\, f(\boldsymbol{x} - \boldsymbol{x}_n, t - t_n)\, dt}{\sum_n \int f(\boldsymbol{x} - \boldsymbol{x}_n, t - t_n)\, dt} \tag{4.21}$$

- Let the kernel function $k(\boldsymbol{x}, \boldsymbol{x}_n), \sum_{n=1}^{N} k(\boldsymbol{x}, \boldsymbol{x}_n) = 1$ have the following form:

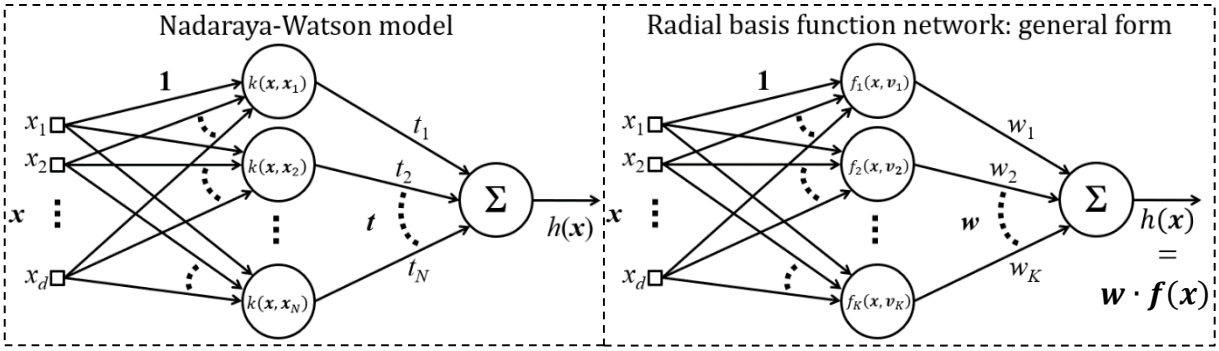
$$k(\boldsymbol{x}, \boldsymbol{x}_n) = \frac{\int t\, f(\boldsymbol{x} - \boldsymbol{x}_n, t - t_n)\, dt}{\sum_n \int f(\boldsymbol{x} - \boldsymbol{x}_n, t - t_n)\, dt} \tag{4.22}$$

- Then (4.21) can be expressed as kernel regression with $\boldsymbol{k}(\boldsymbol{x})$ of Eq. (4.8) and $\boldsymbol{t}$ of Eq. (4.5):

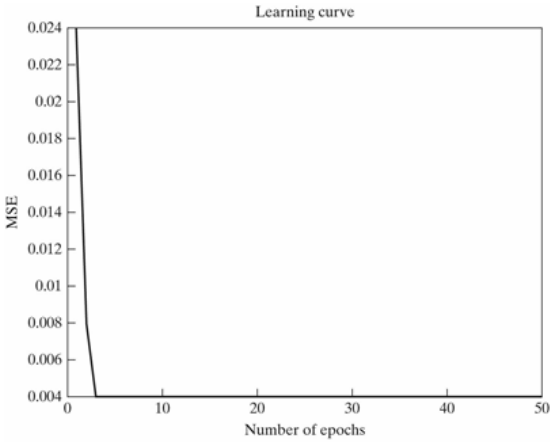
$$h(\boldsymbol{x}) = \sum_{n=1}^{N} k(\boldsymbol{x}, \boldsymbol{x}_n)\ t_n = \boldsymbol{k}(\boldsymbol{x}) \cdot \boldsymbol{t} \tag{4.22}$$

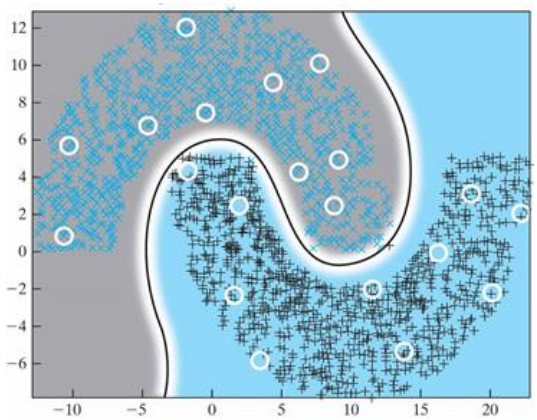
RBF networks for regression and classification
- Input layer: Distribution nodes for the components of input vector $\boldsymbol{x} \in \mathbb{R}^d$
- Hidden layer: Nonlinear transform using RBF centred at every $\boldsymbol{x}_n \in S$
  - E.g. using the Gaussian RBF with distinct standard deviations: $f_n(\boldsymbol{x}) = \exp\left(-\frac{1}{2\sigma_n^2}\|\boldsymbol{x} - \boldsymbol{x}_n\|^2\right)\ \forall n \in [N]$
  - Layer has no weights: equally weighted distribution of the components of $\boldsymbol{x}$
  - For larger data sets: RBF should be centred to K prototype vectors $\boldsymbol{v}_j$ instead to N vectors $\boldsymbol{x}_n$
    - Number of hidden neurons: $K < N$
    - Unsupervised training with the ***k*-means** or **Fuzzy c-means algorithm** to find $\boldsymbol{v}_j$

- Output layer: Weighted sum of RBF output values
  - Only one output neuron with linear activation function for regression tasks, or
  - Only one output neuron with step function for classification tasks
  - Supervised training → Recursive least squares minimisation (RLS) to find $\boldsymbol{w}$: see section 4.6
- **Example plots** [Haykin S (2008) *Neural Networks and Learning Machines*. 3rd ed., Prentice Hall]



- RBF net, $\boldsymbol{x} \in \mathbb{R}^2$, $\mathcal{S}$ is nonlinearly separable, $N = 3000$; number of prototype vectors: $K = 20$





4.6 Recursive Least Mean Squares Minimisation (RLMS)

- Training method of the weight vector of the output neuron of an RBF network
- LS estimation (3.40) according to the

  maximum likelihood principle: (1)

  - ML is an unbiased estimation: (2)
- In the same way the approach is formulated here:

(4.24)

  - Let the number of centroid vectors localised by K-means be K
  - Then we have the $K \times K$ **auto-correlation matrix** of the output signals of the hidden layer neurons

$$\hat{\boldsymbol{R}}_{XX}(i) = \sum_{n=1}^{i} \boldsymbol{f}(\boldsymbol{x}_n)\ \boldsymbol{f}^{\mathrm{T}}(\boldsymbol{x}_n)\ ;\ \boldsymbol{f}(\boldsymbol{x}_n) = \begin{pmatrix} f(\boldsymbol{x}_n, \boldsymbol{x}_1) \\ f(\boldsymbol{x}_n, \boldsymbol{x}_2) \\ \vdots \\ f(\boldsymbol{x}_n, \boldsymbol{x}_N) \end{pmatrix};\ f(\boldsymbol{x}_n, \boldsymbol{x}_k) = \exp\left(-\frac{1}{2\sigma^2}\|\boldsymbol{x}_n - \boldsymbol{x}_k\|^2\right) \tag{4.25}$$

- The following applies to the discrete cross-correlation function between the target variable and the output signals of the hidden layer neurons, i.e. their **cross-correlation vector**:

$$\hat{\boldsymbol{r}}_{XT}(i) = \sum_{n=1}^{i} \boldsymbol{f}(\boldsymbol{x}_n)\ t_n \tag{4.26}$$

- Sought: unknown weight vector $\hat{\boldsymbol{w}}(i)$

- **LS solution** by inverting $\widehat{\boldsymbol{R}}_{XX}$ is often **expensive** and numerically **not stable**
- Therefore, a **recursive approach** is preferred, so that one iteratively approaches an optimal solution
  - Reformulation of the cross-correlation vector

    (3)

  - Recursive approach for the auto-correlation matrix:

    (4)

  - The error term $e(i)$ is called **innovation**, because it drives iterative modifications
  - A priori estimation of the error term

    (5)

  - For the cross-correlation vector it follows

    (6)

  - Put into the initial equation (4.24) the following results

    (7)

  - From this the iteration formula for the weight vector follows:

(4.27)

  - The recursive approach for the auto-correlation matrix is found by applying the theorem for the inverses of composite matrices (see exercise)
  - The following comes out:

(4.28)

- Introduction of two simplifying abbreviations
  - **State-error covariance matrix**: $\hat{P}(i) = \hat{R}_{XX}{}^{-1}(i-1) = \frac{1}{\sigma_{\tilde{w}}^2}\mathbb{E}[(\boldsymbol{w}-\widehat{\boldsymbol{w}})(\boldsymbol{w}-\widehat{\boldsymbol{w}})^{\mathrm{T}}]$
  - **Gain vector**: $\widehat{\boldsymbol{g}}(\boldsymbol{x}_n) = \hat{R}_{XX}{}^{-1}(i-1)\,\boldsymbol{f}(\boldsymbol{x}_n) = \hat{P}(i)\,\boldsymbol{f}(\boldsymbol{x}_n)$
- This gives the recursive approach of the sought weight vector as follows:

(4.29)

Summary of the RLS algorithm

Given: $S = \{(\boldsymbol{x}_n, t_n) | \boldsymbol{x}_n \in \mathcal{X} \subset \mathbb{R}^d, t_n \in \mathcal{T} \subset \mathbb{R}\}$

Initialisation: $i = 0\,,\ \hat{\boldsymbol{w}}(0) = \boldsymbol{0}\,,\ P(0) = \frac{1}{\lambda}\boldsymbol{I}$

Computations: Increment i , set $n = i$

(step 1) $$\boldsymbol{P}(i) = \boldsymbol{P}(i-1) - \frac{\boldsymbol{P}(i-1)\,\boldsymbol{f}(\boldsymbol{x}_n)\,\boldsymbol{f}^{\mathrm{T}}(\boldsymbol{x}_n)\,\boldsymbol{P}(i-1)}{1 + \boldsymbol{f}^{\mathrm{T}}(\boldsymbol{x}_n)\,\boldsymbol{P}(i-1)\,\boldsymbol{f}(\boldsymbol{x}_n)}$$

(step 2) $$\boldsymbol{g}(i) = \boldsymbol{P}(i)\,\boldsymbol{f}(\boldsymbol{x}_n)$$

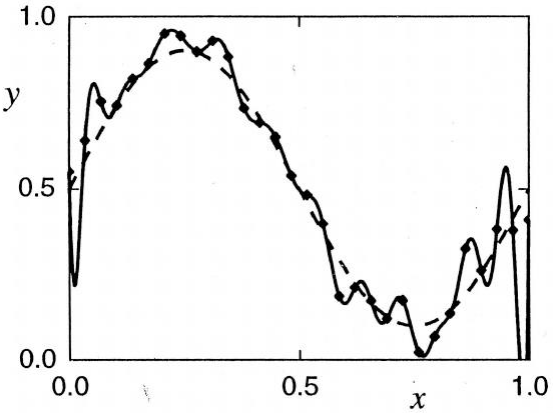
(step 3) $$e(i) = t_n - \hat{\boldsymbol{w}}(i-1) \cdot \boldsymbol{f}(\boldsymbol{x}_n)$$

(step 4) $$\hat{\boldsymbol{w}}(i) = \hat{\boldsymbol{w}}(i-1) + \boldsymbol{g}(i)\,e(i)$$

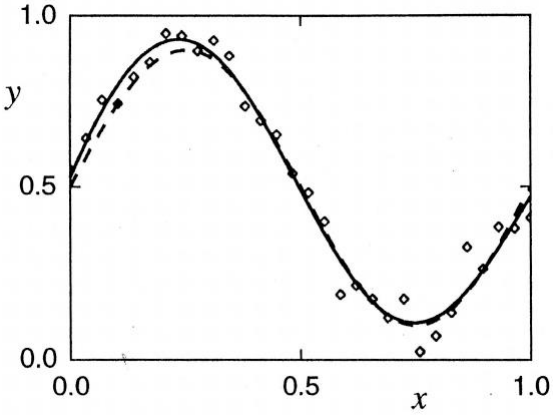
Example [Bishop C (1995) *Neural Networks for Pattern Recognition*. 1st ed., Oxford University Press]

- Training set: $S = \{(x_n, t_n) | x_n \in \mathbb{R}, t_n \in \mathbb{R}, n \in [N], N = 30\}$ with equidistantly selected $x_n \in (0,1)$ and with target function: $t_n = 0.5 + 0.4\sin(2\pi x_n) + \varepsilon_n\,,\ \varepsilon_n \sim \mathcal{N}(0, 0.05)$

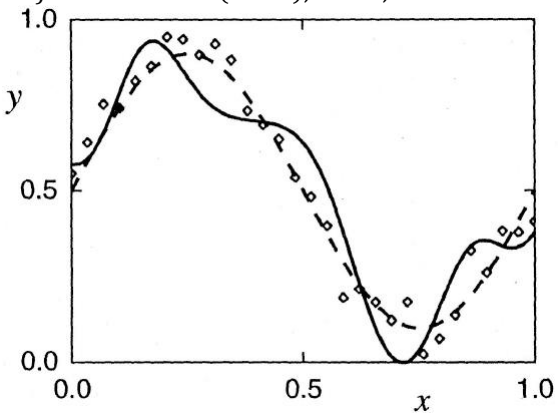
a) LS solution $(\lambda = 0), K = N, \sigma = 0.067 \approx 2d_{avg}$



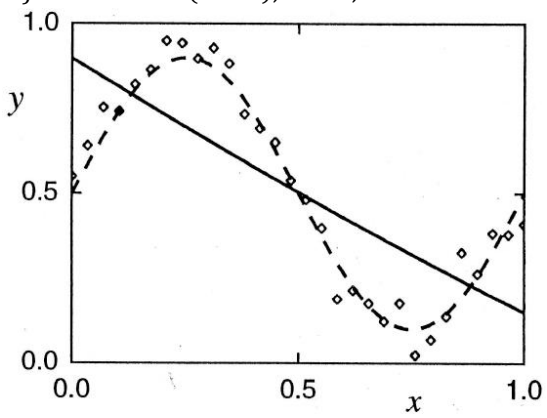
b) RLS solution $(\lambda > 0), K = 5, \sigma = 0.4 \approx 2d_{avg}$



b) RLS solution $(\lambda > 0), K = 5, \sigma = 0.08$



d) RLS solution $(\lambda > 0), K = 5, \sigma = 10$



Final remarks

- RLS aims at minimising the following objective function:

$$J(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{i}[t_n - \hat{\boldsymbol{w}}(n) \cdot \boldsymbol{f}(\boldsymbol{x}_n)^2] + \frac{\lambda}{2}\,\|\hat{\boldsymbol{w}}(n)\|^2 \tag{4.30}$$

- Hidden layer centroids have to be adapted by unsupervised training
  - E.g. *k*-means or Fuzzy *c*-means for adaptation of prototype vectors as centroids
  - Parameters of the RBF kernel functions must be pre-determined or to be optimised empirically
    - In case of Gaussian RBF only the kernel width σ must be determined
    - [Lowe, 1989] suggested to determine σ uniformly for all neurons as follows: $\sigma = \frac{d_{max}}{\sqrt{2K}}$ where d_{max} is the maximum distance between all centroids